

Solutions last updated: Thu, May 21, 2026

PRINT Your Name: _____

PRINT Your Student ID: _____

PRINT Student name to your left: _____

PRINT Student name to your right: _____

You have 170 minutes. There are 7 questions of varying credit. (100 points total)

Question:	1	2	3	4	5	6	7	Total
Points:	17	16	9	14	16	18	10	100

We reserve the right to deduct points for failing to follow the marking directions below:

For questions with **circular bubbles**, you may select only one choice.

☐ Unselected option (Completely unfilled)

☐ Don't do this (it will be graded as incorrect!)

☒ Only one selected option (completely filled)

For questions with **square checkboxes**, you may select one or more choices.

☐ You can select

☐ multiple squares

☒ Don't do this (it will be graded as incorrect!)

Anything you write outside the answer boxes or you ~~cross-out~~ will not be graded. If you write multiple answers, your answer is ambiguous, or the bubble/checkbox is not entirely filled in, we will count the lowest-scoring interpretation.

Read the honor code below and sign your name.

By signing below, I affirm that all work on this exam is my own work. I have not referenced any disallowed materials, nor collaborated with anyone else on this exam. I understand that if I cheat on the exam, I may face the penalty of an "F" grade and a referral to the Center for Student Conduct.
--

SIGN your name: _____

Q1 Potpourri

(17 points)

Q1.1 (1 point) Fill in the blank with the best option: Running BFS (breadth-first) tree search and running BFS graph search on the same graph will _____ return paths of different length.

☐ (A) Always

☐ (B) Sometimes

☒ (C) Never

Solution: BFS tree search and BFS graph search are guaranteed to find the shortest path to the goal state, where shortest is defined in terms of the length of the path (not cost).

For Q1.2 to Q1.3, let h_1 and h_2 be admissible heuristics.

Q1.2 (1 point) True or false: A* tree search using h_1 may find a lower-cost solution than A* tree search using h_2 .

☐ (A) True

☒ (B) False

Solution: A* tree search with admissible heuristics is guaranteed to find the lowest cost solution.

Q1.3 (1 point) Let h_3 be a heuristic such that $h_3(s) = \min(h_1(s), h_2(s))$ for all states s .

True or false: h_3 is admissible.

☒ (A) True

☐ (B) False

Solution: If h_1 and h_2 are admissible, then for any given state we know $0 \leq h_1(s) \leq h^*(s)$ and $0 \leq h_2(s) \leq h^*(s)$ where h^* is the true cost. Then, $0 \leq \min(h_1(s), h_2(s)) \leq h^*(s)$

Q1.4 (1 point) Fill in the blank with the best option: Minimax with alpha-beta pruning will _____ return a different best action than minimax without alpha-beta pruning if the tree is unbalanced.

☐ (A) Always

☒ (B) Sometimes

☒ (C) Never

Solution: Intended answer was “Never”. Alpha-beta pruning is strictly an optimization technique. It is guaranteed to return the exact same minimax value and root action as unpruned Minimax.

We also accepted “Sometimes” if using a modified version of the algorithm.

Q1.5 (1 point) Fill in the blank with the best option: If we change all the minimizer nodes in a minimax tree to chance nodes that pick actions uniformly at random, the value at the root will _____ increase.

☐ (A) Always

☒ (B) Sometimes

☐ (C) Never

Solution: The minimum of a set of values is always lower than the average, unless all the values are the same.

(Question 1 continued...)

For Q1.6 to Q1.7, consider the Gridworld below. In states A and E , the only action available is “Exit”, which earns the +100 or +1 reward, respectively. In states B , C , and D , the actions are “Left” and “Right” which are successful 100% of the time. There is no reward other than the +100 or +1.

+100				+1
A	B	C	D	E

Q1.6 (2 points) When running value iteration on this Gridworld, what is the smallest k for which $V_k(s) = V_{k+1}(s)$ for all states $s \in \{A, B, C, D, E\}$?

Assume that the discount factor is $\gamma = 1$ and that $V_0(s) = 0$ for all states $s \in \{A, B, C, D, E\}$.

- ☐ A 0 ☐ B 1 ☐ C 2 ☐ D 3 ☒ E 4 ☐ F 5

Solution: At V_0 , all states have a value of 0. At V_1 , $V_1(A) = 100$ and $V_1(E) = 1$. For the next iteration, $V_2(B) = 100$ and $V_2(D) = 1$. Then, $V_3(C) = 100$. At V_4 , state D sees that it has a higher reward if the agent goes left, so $V_4(D) = 100$. No values change at V_5 , so $k = 4$.

Q1.7 (2 points) For what discount factor are the actions “Left” and “Right” equally good when in state D ? Your answer should be a number.

0.1

Solution: We solve for $\gamma^3 \cdot 100 = \gamma \cdot 1$ to get $\gamma = 0.1$

Q1.8 (1 point) The general Bellman update $V'(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V(s)]$ requires $O(SA)$ time to compute for one state.

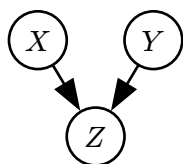
Now, suppose the MDP is deterministic, with the probabilistic model $T(s, a, s')$ replaced by the function $\text{Result}(s, a)$. What is the big-O runtime now to compute the Bellman update for one state? Answer in big-O notation.

$O(A)$

Solution: We no longer need to sum over all states as there is only one s' with non-zero probability, and we can access it using the Result function.

(Question 1 continued...)

Q1.9 (1 point) Andrew is running Gibbs sampling on the following Bayes Net where each variable can take on the value 0 or 1. The probability table for Z is provided below (note that the table is equivalent to saying that $Z = X \text{ XOR } Y$):



X	Y	Z	$P(Z X, Y)$
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

The current state of the Gibbs sampling process is $X = 0, Y = 1, Z = 1$.

We choose to re-sample X . What is the probability that X will be re-sampled as 0?

- ☐ (A) 0 ☒ (C) 1
☐ (B) 0.5 ☐ (D) None of the above

Solution: $X = 1, Y = 1, Z = 1$ is impossible, so X must be re-sampled as 0.

Q1.10 (1 point) True or false: In a Hidden Markov Model (HMM), particle filtering generally gives more accurate values for $P(X_t | e_{1:t})$ than the forward algorithm.

- ☐ (A) True ☒ (B) False

Solution: Particle filtering is an approximate algorithm while the forward algorithm is exact inference.

Q1.11 (2 points) Which of the following represent *advantages* of transformer models covered in class *over* trigram models?

- ☐ (A) Allow adjacent-word correlations to be captured
☒ (B) Allow distant-word correlations to be captured
☐ (C) Allow words that are more frequent in the training data to be given higher probabilities
☐ (D) Can assign nonzero probabilities to sentences never seen in the training data
☐ (E) None of the above

Solution: Attention mechanisms allow for distant-word correlations.

(Question 1 continued...)

Q1.12 (1 point) Isabella wants to create an AI assistant for students in CS188. She first pre-trains a language model on transcripts of CS188 lectures. When she evaluates it on a test set of CS188 lectures, she finds that the model achieves high accuracy on predicting the next token.

Which of the following is the best next step for Isabella to take?

- ☐ (A) Use a tokenizer that is trained on lectures from more courses, not just CS188
- ☒ (B) Fine-tune the model on helpful human conversations
- ☐ (C) Train the model on CS188 lectures using self-supervised learning for additional epochs
- ☐ (D) Increase the size of the weight matrices used in the attention layers

Solution: Isabella has done a good job pre-training, but needs to fine-tune the model to be helpful in order to be an AI assistant. Lectures typically contain few extended Q/A pairs and conversations, and lecturers often say things like “Happy to take that one offline”.

Q1.13 (1 point) Kanav is using a word bigram model. Select the operator to fill in the blanks that makes the following equation true under this model:

$$P(\text{"Pac-Man wins"}) = P(\text{"Pac-Man"} \mid \text{START}) \text{ ______ } P(\text{"wins"} \mid \text{"Pac-Man"}) \text{ ______ } P(\text{STOP} \mid \text{"wins"})$$

☐ (A) + ☐ (B) − ☒ (C) × ☐ (D) ÷

Solution: We can apply the chain rule of probabilities here (simplifying with the bigram assumption). This means we need to multiply the probabilities above together.

Q1.14 (1 point) Which of the following is the most suitable method to prevent overfitting in a decision tree?

- ☐ (A) Minimizing the information gain at the root node
- ☐ (B) Maximizing the average entropy of the label distributions at the leaves of the tree
- ☐ (C) Removing a random subset of data points from the training data
- ☒ (D) Limiting the depth of the decision tree

Solution: Making the decision tree more compact means it is less likely to fit the training data exactly and simply memorize it.

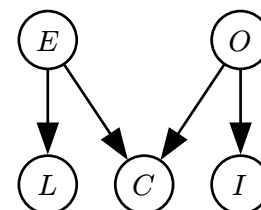
Q2 Bayes Nets: Printer Problems

(16 points)

It's the night before the CS188 final exam, and Niklas and Aly are printing copies of the exam. We use the Bayes Net to the right to model whether the printer will print a single exam correctly.

The variables in our Bayes Net are

- E : is the toner empty?
- L : is the empty toner light on?
- O : is the printer out of paper?
- I : is the paper indicator on?
- C : does the printer print the exam correctly?



Assume each variable is binary (e.g., E can take on values e or $\neg e$).

Q2.1 (2 points) Select all of the following variables that are guaranteed to be **independent** of L given E .

☒ A C

☒ C I

☐ B O

☐ D None of the above

Solution: In a Bayes Net, a variable is independent of all its non-descendants given its parents. C , O , and I are non-descendants of L , and we are given the parents of L (i.e. E).

Q2.2 (2 points) Select all of the following variables that are guaranteed to be **dependent** on I given C .

☐ A O

☐ C L

☐ B E

☐ D None of the above

Solution: None of the above. Arrows in the Bayes Net don't necessarily encode dependence.

Q2.3 (2 points) How many entries are there in total across all probability tables in the Bayes Net? *Ignore* entries whose values are determined by the sum-to-1 constraint. E.g., $P(X)$ for a three-valued variable X is just two entries because the third is determined by the first two.

10

Solution: The number of entries in a CPT is the product of the parent domain sizes and the child domain size minus 1. The CPTs we have are $P(E)$, $P(L | E)$, $P(C | E, O)$, $P(O)$, and $P(I | O)$. The total number of entries across all CPTs is $(2 - 1) + 2^{2-1} + 2^2(2 - 1) + (2 - 1) + 2(2 - 1) = 10$.

(Question 2 continued...)

Q2.4 (2 points) Niklas wants to calculate $P(C \mid i)$ using variable elimination and decides to eliminate the variable O first. Which new factor f does this create?

- | | | | |
|-----------------------------------|--------------------------------------|---|--|
| ☐ Ⓐ $f(O)$ | ☐ Ⓓ $f(O, C)$ | ☒ Ⓒ $f(E, C, i)$ | ☐ Ⓘ $f(L, E, C, i)$ |
| ☐ Ⓑ $f(C)$ | ☐ Ⓔ $f(C, i)$ | ☐ Ⓗ $f(O, E, C)$ | ☐ Ⓚ $f(L, O, E, C, i)$ |
| ☐ Ⓒ $f(E, C)$ | ☐ Ⓕ $f(O, C, i)$ | ☐ Ⓛ $f(O, E, C, i)$ | ☐ Ⓜ None of the above |

Solution: Join the factors with O : $P(C \mid E, O)$, $P(O)$, and $P(i \mid O)$ to get a factor with C , E , O , and i . If we sum out O , we get $f(E, C, i)$.

Q2.5 (1 point) Which of the following orders is a valid visitation order for applying prior sampling to this Bayes net?

- | | | | |
|--|--|---|--|
| ☐ Ⓐ L, E, C, O, I | ☐ Ⓑ C, E, I, L, O | ☒ Ⓒ E, L, O, C, I | ☐ Ⓓ None of the above |
|--|--|---|--|

Solution: Must be in topological order.

Q2.6 (1 point) Fill in the blank with the best option: In the limit as the number of samples tends to infinity, if we run rejection sampling to estimate $P(E \mid l)$, we will ____ reject more samples than if we run rejection sampling to estimate $P(E \mid l, \neg c)$.

- | | |
|-----------------------------------|---|
| ☐ Ⓐ Always | ☒ Ⓒ Never |
| ☐ Ⓑ Sometimes | ☐ Ⓓ Cannot tell with the provided information |

Solution: While estimating $P(E \mid l, \neg c)$, you will reject samples that have a $\neg l$ or a c , and in the limit, this will be more samples than have a $\neg l$ only.

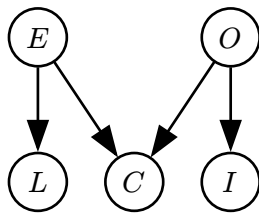
(Question 2 continued...)

Q2.7 (2 points) For this part only, we find that there's an app that can tell us when there is a problem with the toner and/or paper. We want to include this in our Bayes Net.

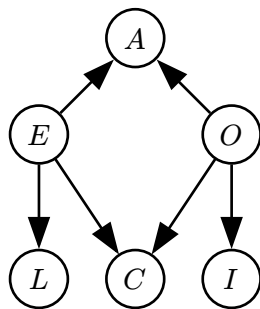
Let the app variable be represented by A . A can take on one of two values:

- a : signals that there is a problem (toner is empty and/or paper is out)
- $\neg a$: signals that there is not a problem (toner is not empty and paper is not out)

Add the appropriate nodes and edges to the Bayes Net below.



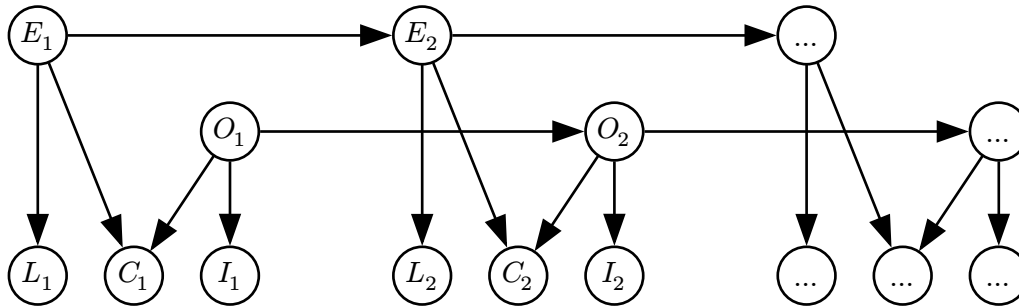
Solution:



Because A is a signal for E and O , we draw arrows from E and O to A .

(Question 2 continued...)

For Q2.8 to Q2.9, we use the following dynamic Bayes Net to model printing out multiple different copies of the exam, where each time step represents printing out one copy. We show the first two time steps, and the pattern repeats after that:



Q2.8 (2 points) For this part only, we use likelihood weighting to compute $P(E_2 \mid L_2 = l, I_2 = \neg i)$.

We generate the sample:

$(E_1 = \neg e, L_1 = \neg l, O_1 = o, I_1 = i, C_1 = c, E_2 = e, L_2 = l, O_2 = \neg o, I_2 = \neg i, C_2 = \neg c)$

Select the set of probability distributions that we need to compute the *weight* of this sample (not the sample itself).

- ☐ A $P(L_t = l)$
☐ D $P(I_t = i)$
☐ G $P(E_1)$
- ☒ B $P(L_t = l \mid E_t = e)$
☐ E $P(I_t = i \mid O_t = o)$
☐ H $P(E_{t+1} \mid E_t = \neg e)$
- ☐ C $P(L_t = l \mid E_t = \neg e)$
☒ F $P(I_t = i \mid O_t = \neg o)$
☐ I None of the above

Solution: We need to compute the likelihood of the evidence, and we can multiply together the likelihoods of the individual evidence l and $\neg i$ given the structure of the Bayes Net.

Q2.9 (2 points) Consider the following probability table for $P(E_{t+1} \mid E_t)$:

E_t	E_{t+1}	$P(E_{t+1} \mid E_t)$
e	e	1
e	$\neg e$	0
$\neg e$	e	0.01
$\neg e$	$\neg e$	0.99

As the time step t approaches ∞ , what does $P(E_t = e)$ approach?

- ☐ A 0
 ☐ C 0.5
 ☒ E 1
- ☐ B 0.01
 ☐ D 0.99
 ☐ F None of the above

Solution: We look at the stationary distribution which would be solved by $\begin{pmatrix} 1 & 0.01 \\ 0 & 0.99 \end{pmatrix} \begin{pmatrix} P(E_t=e) \\ 1-P(E_t=e) \end{pmatrix} = \begin{pmatrix} P(E_t=e) \\ 1-P(E_t=e) \end{pmatrix}$ which will give us that $P(E_t = e) = 1$. If we apply the transition to this distribution, we will get the same distribution, so it is the stationary distribution.

Q3 Please Don't Click on the Mines!**(9 points)**

Advika is playing a game on the following 3 x 3 board:

2	x_2	x_3
2	x_5	x_6
x_7	x_8	1

Some squares contain mines, and Advika has to avoid clicking on them. The squares labeled x_2, x_3, x_5, x_6, x_7 and x_8 have not yet been clicked on, and their contents are unknown.

Advika has already clicked on the squares marked 2, 2, and 1. (Thank goodness she is still alive!) Clicking on a square results in either death (if there is a mine) or reveals the number of mines in horizontally, vertically, and diagonally adjacent squares. For example, the 1 in the lower-right corner of the board means that there is 1 mine total under the three adjacent squares labeled x_5, x_6 , and x_8 .

To help her decide where it is safe to click next, Advika formulates a CSP. The variables x_i can take on values in $\{0, 1\}$ where $x_i = 1$ means there is a mine in that square, and $x_i = 0$ means there is not. (Note that the CSP applies only to this turn; after she clicks, Advika will need to update the CSP accordingly.) The constraints in the CSP arise from the mine-count information revealed by previous clicks.

Q3.1 (1 point) Which of the following constraints correctly reflect the current situation? Select all that apply.

☒ **A** $x_2 + x_5 = 2$

☐ **C** $x_6 + x_8 = 1$

☐ **B** $x_2 + x_5 + x_7 + x_8 = 1$

☐ **D** None of the above

Solution: The 2 in the upper-right corner is answer choice A. B has the wrong number of mines. C is missing x_5 .

(Question 3 continued...)

Q3.2 (2 points) Advika is running backtracking search with forward checking. Her search assigns variables in the order $x_2, x_5, x_3, x_6, x_7, x_8$. Values are assigned in lexicographic order (0 then 1).

She assigns $x_2 = 0$. What happens next?

- ☐ (A) Advika assigns $x_5 = 0$ because x_5 is the next variable and 0 is the first value to try.
- ☐ (B) Advika assigns $x_5 = 0$ because the arc from $x_5 \rightarrow x_2$ is consistent if $x_5 = 0$.
- ☒ (C) Advika backtracks because there are no valid assignments for x_5 .
- ☐ (D) Advika backtracks because the depth-first search will run into a cycle.
- ☐ (E) None of the above

Solution: Advika backtracks because forward checking shows no valid values for x_5 .

Q3.3 (2 points) For this subpart only, let us consider general instances of this family of CSPs and suppose we add a new constraint: $\sum_i x_i \leq N$ where N is the maximum number of mines allowed on the grid. Which of the following are true?

- ☐ (A) The number of satisfying solutions to the CSP may increase.
- ☐ (B) The number of satisfying solutions to the CSP may stay the same.
- ☒ (C) The number of satisfying solutions to the CSP may decrease.
- ☐ (D) None of the above

Solution: Adding more constraints will never increase the number of satisfying solutions.

Q3.4 (2 points) Backtracking search yields a solution to the CSP where $x_3 = 0$. Can Advika be certain that the corresponding square is safe?

- ☐ (A) Yes, because backtracking yields correct solutions to CSP problems.
- ☐ (B) Yes, because backtracking search works with higher-order constraints.
- ☐ (C) Yes, because of another reason not listed above
- ☐ (D) No, because backtracking does not always yield correct solutions to CSP problems.
- ☐ (E) No, because backtracking only works with binary constraints.
- ☒ (F) No, because of another reason not listed above.

Solution: No, because backtracking will return *some* solution to the CSP; but for a square to be safe it has to be mine-free in *all* solutions to the CSP. If there is *any* solution with a mine, that means it's *possible* the square has a mine given all the information so far.

(Question 3 continued...)

Q3.5 (2 points) Advika notices that she can split this CSP into two disjoint CSPs that can be solved independently: CSP A and CSP B. She combines the solutions to CSP A and CSP B to generate a solution to the original CSP.

CSP A contains the x_5 variable. What are the other variables in CSP A?

☒ A x_2

☒ D x_7

☐ B x_3

☒ E x_8

☒ C x_6

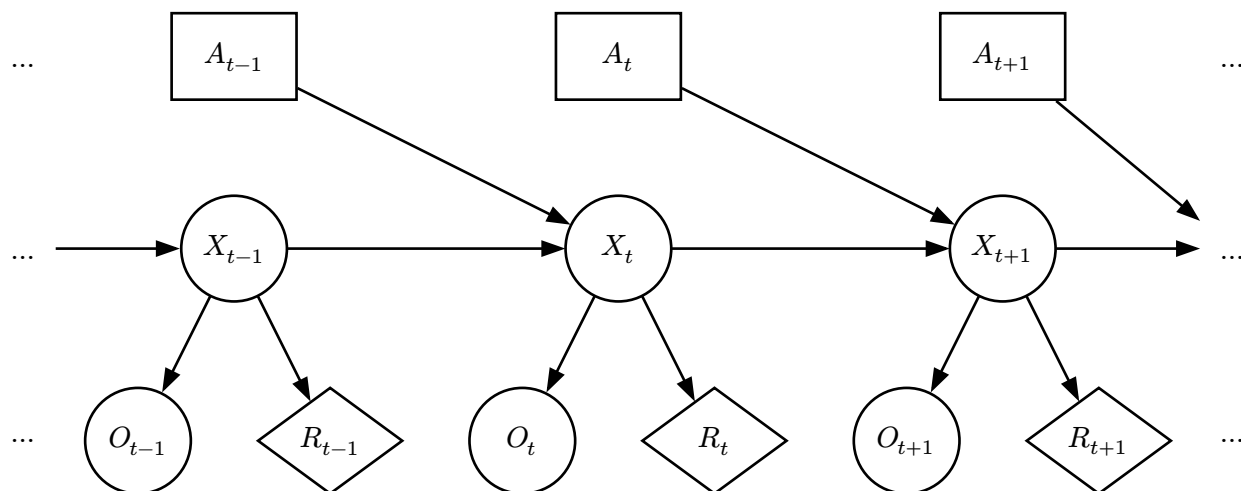
☐ F None of the above

Solution: The constraints of the graph are: $x_2 + x_5 = 2$, $x_2 + x_5 + x_7 + x_8 = 2$, and $x_5 + x_6 + x_8 = 1$. All variables that are involved in constraints with x_5 must be in the same CSP as x_5 . x_3 is not in any constraints, so it can be in its own CSP. Note that the set of *all* solutions to the CSP is the Cartesian product of the sets of solutions to the sub-CSPs.

Q4 Dynamic Decision Network

(14 points)

A dynamic decision network is dynamic Bayes net extended with action and reward nodes. In the example below, X_t is the state of the environment at time t , A_t is the action, R_t is the reward, and O_t is the observation. Note the indexing of variables: the edge goes from A_{t-1} to X_t . By convention, O_t is observed before A_t is chosen.



Suppose we want to maximize the sum of rewards $\sum_{t'=0}^T R_{t'}$ and we're currently at time step t (we've taken actions $A_{1:t-1}$ and observed $O_{1:t}$). Select the conditions under which each equation holds.

Q4.1 (1 point) $VPI(O_{t+1}|X_{t+1}) = 0$

- ☒ (A) Always ☐ (B) Sometimes ☐ (C) Never

Solution: The VPI of a future observation comes from added information about the associated state, but if we are given the value of that state, the observation adds nothing.

Q4.2 (1 point) $VPI(X_t, O_{t+1}) < VPI(X_t)$

- ☐ (A) Always ☐ (B) Sometimes ☒ (C) Never

Solution: Since the expected value of information is nonnegative, adding an observation cannot have negative value.

Q4.3 (1 point) $VPI(X_t, X_{t+1}) > VPI(X_t)$

- ☐ (A) Always ☒ (B) Sometimes ☐ (C) Never

Solution: Adding the future state can have positive value, but it can also have zero value if (say) the transition model is deterministic and identical for all actions.

(Question 4 continued...)

Let the *belief state* $\mathbf{f}_t = P(X_t | o_{1:t}, a_{1:t-1})$ be a distribution representing the agent's belief over the hidden state at timestep t given the observations up to this point as well as all previous actions.

Q4.4 (2 points) Which of the following elements are required for us to compute a belief update: computing $\mathbf{f}_t(X_t)$ from $\mathbf{f}_{t-1}(X_{t-1})$?

- ☐ A $P(X_t | X_{t-1}, a_{t-1})$ ☐ D $P(o_t | X_t)$
☐ B $P(X_t | X_{t-1}, a_t)$ ☐ E $P(X_t | o_t)$
☐ C $P(X_t | X_{t-1})$ ☐ F None of the above

Solution: $\mathbf{f}_t(x_t) = \alpha P(o_t | x_t) \sum_{x_{t-1}} P(x_t | x_{t-1}, a_{t-1}) \mathbf{f}_{t-1}(x_{t-1})$

Q4.5 (2 points) An optimal action a_t at timestep t is an action that maximizes the expected sum $\mathbb{E}[\sum_{t'=t}^T r_{t'} | o_{1:t}, a_{1:t-1}]$ of future rewards given previous observations and actions. In our dynamic decision network, knowing just $\mathbf{f}_t$ is sufficient to take an optimal action at timestep t . Why is this?

- ☐ A It stores the most likely hidden state
☐ B The future observations are conditionally independent of future actions, given $\mathbf{f}_t$
☒ C The distribution over future trajectories is conditionally independent of previous actions and observations, given $\mathbf{f}_t$
☐ D The belief state $\mathbf{f}_t$ is computed recursively in a way that allows you to reconstruct all previous observations $o_{1:t}$ and actions $a_{1:t-1}$
☐ E The current observation o_t contains all information relevant to future rewards
☐ F None of the above

Solution: Because of the Markov property, the belief state $\mathbf{f}_t$ summarizes all information from the action-observation history that is relevant for predicting future states (and thus rewards). Therefore, conditioned on $\mathbf{f}_t$, the distribution over future trajectories does not depend on the earlier observations $o_{1:t}$ or actions $a_{1:t-1}$.

Q4.6 (2 points) Suppose we want to use our dynamic decision network to model a gridworld environment in which we receive a reward of +1 every time we take the $A=\text{MoveNorth}$ action. How do we have to change the graphical model to model this?

- ☐ A Add an edge from O_t to X_{t+1} ☐ D Add an edge from A_{t-1} to O_t
☐ B Add an edge from O_t to R_t ☐ E Add an edge from R_{t+1} to X_t
☐ C Add an edge from R_t to A_{t-1} ☐ F None of the above

Solution: None of the above is correct. Action now needs to influence the reward, so A_{t-1} to R_t .

(Question 4 continued...)

Q4.7 (2 points) Suppose we want to use our dynamic decision network to model a gridworld environment where the $A = \text{TakeMeasurement}$ action reveals our current state. How do we have to change the graphical model to model this?

- ☐ A Add an edge from O_t to X_{t+1} ☒ D Add an edge from A_{t-1} to O_t
- ☐ B Add an edge from A_t to X_t ☐ E Add an edge from A_{t-1} to R_t
- ☐ C Add an edge from X_t to A_t ☐ F None of the above

Solution: Previous action now needs to influence the observation (revealing the current state), so A_{t-1} to O_t . If $A_{t-1} = \text{TakeMeasurement}$, the conditional distribution for O_t is deterministic with all mass on the true value of X_t . Connecting to X_t doesn't work because that means the action *sets* the value of the state rather than observes it.

Q4.8 (2 points) In which of the following cases are our observations $o_{1:T}$ guaranteed to not affect the optimal actions $a_{1:T}$?

- ☒ A Observations are independent of the state
- ☐ B Observations are a deterministic function of the states
- ☐ C States are fully observable
- ☒ D State transitions are deterministic and known and x_0 is known
- ☐ E None of the above

Solution:

- If observations are independent of the state, they provide no information $\rightarrow$ no effect on policy.
- Observations being deterministic function of state makes observations maximally informative; fully observable even more so!
- Known initial state x_0 , deterministic transitions, and actions (implicit in policy) are enough to determine state.

Q4.9 (1 point) To estimate the belief state $f_t(X_t)$, we decide to use a particle filter. When is the observation o_t used in the particle filter update?

- ☐ A When propagating particles using the transition model
- ☒ B When weighting particles by likelihood
- ☐ C When sampling initial particles
- ☐ D Observations are not relevant to the particle filter update
- ☐ E None of the above

Solution: Observations are used to weight particles before resampling the hidden state.

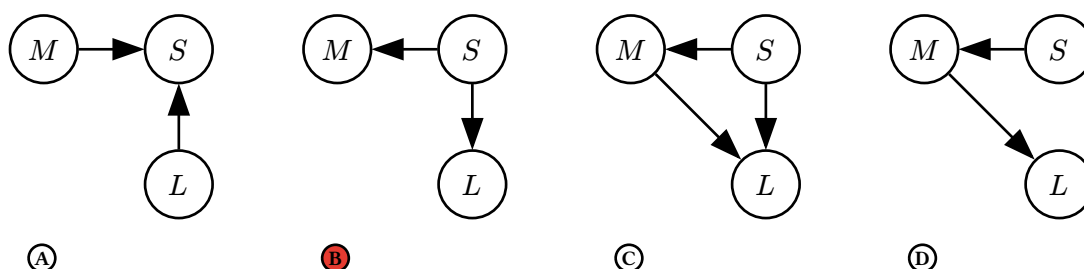
Q5 Naive Bass

(16 points)

You want to classify what species S a fish is. You recently learned that fish with upturned mouths can feed better at the water surface, and downturned mouths are adapted to feed at the bottom. You go to the aquarium and observe four fish, noting their mouth shape M (upturned or downturned), feeding location L (at the top or at the bottom of the tank) and species S (flatfish or bass), shown below. You build a Naive Bayes classifier using M and L , using the four fish as the training dataset D . Initially you use maximum likelihood estimation with no smoothing.

Fish ID	Mouth (M)	Feeding Location (L)	Species (S)
1	upturned	top	bass
2	downturned	bottom	flatfish
3	upturned	bottom	bass
4	downturned	top	bass

Q5.1 (1 point) Which of these Bayes nets corresponds to the Naive Bayes model of this classification problem?



Solution: Naive Bayes assumes all features are conditionally independent given the class variable S , as in B. The extra edge between M and L in C is superfluous. Network A asserts that M and L are absolutely independent, which need not be true. Network D asserts that L is independent of S given M , which need not be true.

Q5.2 (2 points) You spot a fish feeding at the bottom of its tank with an upturned mouth. According to your model, what is the probability that it is a bass?

- ☐ A 0 ☐ B $\frac{1}{4}$ ☐ C $\frac{1}{3}$ ☐ D $\frac{1}{2}$ ☒ E $\frac{2}{3}$ ☐ F $\frac{3}{4}$ ☒ G 1
☐ H None of the above

Solution:

$$P(S|b, u) = \alpha P(b, u|S)P(S) = \alpha P(b|S)P(u|S)P(S) = \alpha \left(\frac{1}{3}\right) \left(\frac{2}{3}\right) \left(\frac{3}{4}\right) = \alpha \left(\frac{1}{6}\right) = \left(\frac{1}{0}\right)$$

(Question 5 continued...)

Q5.3 (2 points) What is the probability your model estimates a fish feeding at the bottom with a **downturned** mouth is a bass?

- ☐ A 0 ☐ C $\frac{1}{9}$ ☒ E $\frac{1}{4}$ ☐ G $\frac{1}{2}$ ☐ I $\frac{3}{4}$
☐ B $\frac{1}{10}$ ☐ D $\frac{1}{12}$ ☐ F $\frac{1}{3}$ ☐ H $\frac{2}{3}$ ☐ J None of the above

Solution:

Here we just need to replace $P(u|S)$ with $P(d|S)$.

$$P(S|b, d) = \alpha P(b, d|S)P(S) = \alpha P(b|S)P(d|S)P(S) = \alpha \left(\frac{1}{3}\right) \left(\frac{1}{3}\right) \left(\frac{3}{4}\right) = \alpha \left(\frac{1}{12}\right) = \left(\frac{1}{4}\right)$$

Q5.4 (3 points) Let P be the model created from the original training data D using maximum likelihood learning without smoothing. Denote the likelihood of data D under model P by $P(D)$. We now consider various ways to make a new Naive Bayes model P' , which is also unsmoothed unless otherwise specified. Select all of the following statements that are true, paying very careful attention to the exact statement of the inequality.

- ☒ A Collect data from *more fish* and combine them with the data in D to create dataset D' . Then train P' on D' to predict S from M and L . It is possible that $P'(D) < P(D)$.
- ☐ B Pick a new feature F and measure its value for each fish in D , to create dataset D' with features L, M , **and** F . Then train P' on D' to predict S from L, M , and F . It is possible that $P'(D') < P(D)$.
- ☐ C Define dataset D' as a copy of D with one of the features removed, then train P' on D' to predict S from the remaining feature alone. It is possible that $P'(D') < P(D)$.
- ☒ D P' has the same features as P and is trained on the same data D , but P' uses Laplace smoothing. It is possible that $P'(D) < P(D)$.
- ☐ E None of the above

Solution: A is true because P maximizes the likelihood of D , but after refitting P' now maximizes the likelihood of D' instead. Because P is the maximum likelihood model, P' can only be the same or worse.

B is true and C is false because Naive Bayes models $P(y, f_{1:i})$ as $P(y) \prod_i P(f_i|y)$. Adding an additional feature adds an additional $P(f_i|y)$ term into the product, while all the other terms are unchanged due to the independence assumption. Thus, this will decrease the likelihood when $P(f_i|y) < 1$. Removing a feature corresponds to removing a factor of $P(f_i|y)$, which cannot decrease the likelihood because $P(f_i|y) \leq 1$.

D is true because the model without smoothing maximizes the likelihood of D , so adding smoothing can decrease the likelihood of D .

(Question 5 continued...)

For the rest of the problem, your model uses data D and Laplace smoothing with strength k .

Q5.5 (1 point) What happens to your estimate of the probability that the next fish you spot is a flatfish, if you increase k from 0 to a positive value X ?

- ☒ (A) Increases ☐ (C) Stays the same ☐ (E) None of the above
☐ (B) Decreases ☐ (D) Depends on X

Solution: If the true counts for bass and flatfish are s and f , the smoothed estimate for the flatfish probability is $\frac{f+k}{s+k+f+k}$. With $k = 0$, the model predicts $P(\text{flatfish}) = \frac{1}{4}$, while increasing k will move the prediction closer to $\frac{1}{2}$.

Q5.6 (2 points) Which of the following are always true as k goes to infinity? Use P to denote the estimated probabilities under the Naive Bayes model.

- ☐ (A) $P(\text{flatfish} \mid \text{downturned, bottom})$ approaches 1.
☐ (B) $P(s, m, l)$ approaches $\frac{1}{2}$ for all values of s, m and l .
☒ (C) $P(l|s)$ approaches $\frac{1}{2}$ for all values of l and s .
☐ (D) The likelihood of the data in D under the model approaches 0.
☐ (E) None of the above

Solution: A is false and C is true because as k goes to infinity, $P(s) \rightarrow \frac{1}{2} \forall s$ and $P(f|s) \rightarrow \frac{1}{2} \forall f, s$, so the model will estimate the same probability for each class no matter the feature values.
B and D are false because all 8 feature and class combinations will be estimated as having equal likelihood of $\frac{1}{8}$.

(Question 5 continued...)

Q5.7 (3 points) Laplace smoothing with strength $k > 0$ assigns a value $P_L(S = s_j) = \frac{N_j + k}{\sum_i [N_i + k]}$ for the probability of the j th species after seeing N_j members of that species, where the sum is over all possible species. Before seeing any data, P_L assigns equal probability $\frac{k}{Mk} = \frac{1}{M}$ to each of M possible species.

Assume (as before) that the only species are bass and flatfish, and that you have not personally seen any fish yet. But you feel very confident, based on chatting with local fishing crews, that bass are about twice as common as flatfish. We can implement this idea by creating *weights* η for each species. Which one of the following formulae for $P_L(s_j)$ properly expresses your prior knowledge?

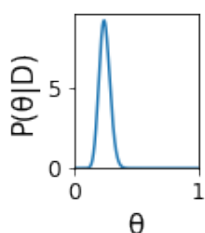
- Ⓐ $\frac{N_j + \eta_j \cdot k}{\sum_i [N_i + \eta_i \cdot k]}$, with $\eta_{\text{bass}} = 1, \eta_{\text{flatfish}} = \frac{1}{3}$, small k
- Ⓑ $\frac{\eta_j \cdot N_j + k}{\sum_i [\eta_i \cdot N_i + k]}$ with $\eta_{\text{bass}} = 1, \eta_{\text{flatfish}} = \frac{1}{3}$, small k
- Ⓒ $\frac{N_j + \eta_j \cdot k}{\sum_i [N_i + \eta_i \cdot k]}$, with $\eta_{\text{bass}} = 1, \eta_{\text{flatfish}} = \frac{1}{2}$, small k
- Ⓓ $\frac{\eta_j \cdot N_j + k}{\sum_i [\eta_i \cdot N_i + k]}$ with $\eta_{\text{bass}} = 1, \eta_{\text{flatfish}} = \frac{1}{2}$, small k
- Ⓔ $\frac{N_j + \eta_j \cdot k}{\sum_i [N_i + \eta_i \cdot k]}$, with $\eta_{\text{bass}} = 1, \eta_{\text{flatfish}} = \frac{1}{3}$, large k
- Ⓕ $\frac{\eta_j \cdot N_j + k}{\sum_i [\eta_i \cdot N_i + k]}$ with $\eta_{\text{bass}} = 1, \eta_{\text{flatfish}} = \frac{1}{3}$, large k
- Ⓖ $\frac{N_j + \eta_j \cdot k}{\sum_i [N_i + \eta_i \cdot k]}$, with $\eta_{\text{bass}} = 1, \eta_{\text{flatfish}} = \frac{1}{2}$, large k
- Ⓗ $\frac{\eta_j \cdot N_j + k}{\sum_i [\eta_i \cdot N_i + k]}$ with $\eta_{\text{bass}} = 1, \eta_{\text{flatfish}} = \frac{1}{2}$, large k

Solution: Believing a priori that there are twice as many bass as flatfish means that when the counts are zero, we want $P(\text{bass}) = \frac{2}{3}$. So we need η for flatfish to be half of the η for bass. The η should multiply k in the formula. (If it multiplies the counts, then it has no effect when the counts are zero.) And we want large k to express high confidence, so that it takes a lot of contrary evidence to shift the prior belief.

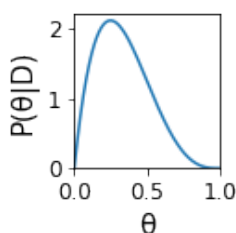
(Question 5 continued...)

Q5.8 (2 points) Bayesian statisticians think of Laplace smoothing as a cheap heuristic approximation to real Bayesian learning! In lecture, we studied Bayesian learning in the context of a finite set of hypotheses (different kinds of bags of candy). But we can also apply it to a continuous family of hypotheses, such as the possible relative frequencies of bass and flatfish. That is, suppose $P(\text{flatfish}) = \theta$ and $P(\text{bass}) = 1 - \theta$. Then Bayesian learning can start with a prior probability $P(\theta)$ and can update that distribution to $P(\theta|D)$ as actual data D on bass and flatfish are observed.

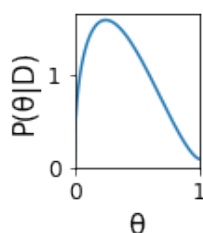
Suppose that you start with a uniform prior $P(\theta)$ over the range $[0,1]$ and then you observe one flatfish and three bass, as shown in the original dataset D . Which of the following best represents $P(\theta|D)$?



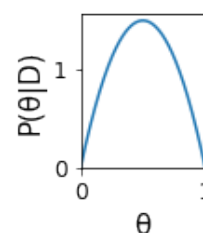
(A)



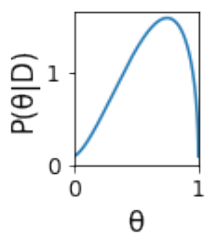
(B)



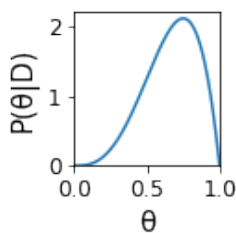
(C)



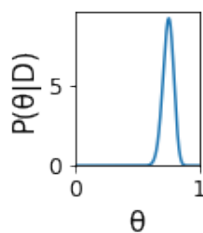
(D)



(E)



(F)



(G)

Solution: A uniform prior suggests you have no idea about the relative frequencies. After 1 flatfish and 3 bass, you should still be quite uncertain about θ , but your estimate should have most of its probability mass close to $1/4$, which leaves B and C. It is not possible for θ to be 1 or 0 because we observed both flatfish and bass, thus C is incorrect.

Q6 RL: Grid Racetrack

(18 points)

Oski is training a Reinforcement Learning agent to play the classic paper-and-pencil game Racetrack. The game is played on a 2D discrete grid. At any time step, the car's state is fully defined by its exact coordinates and velocity vector: $s = (x, y, v_x, v_y)$.

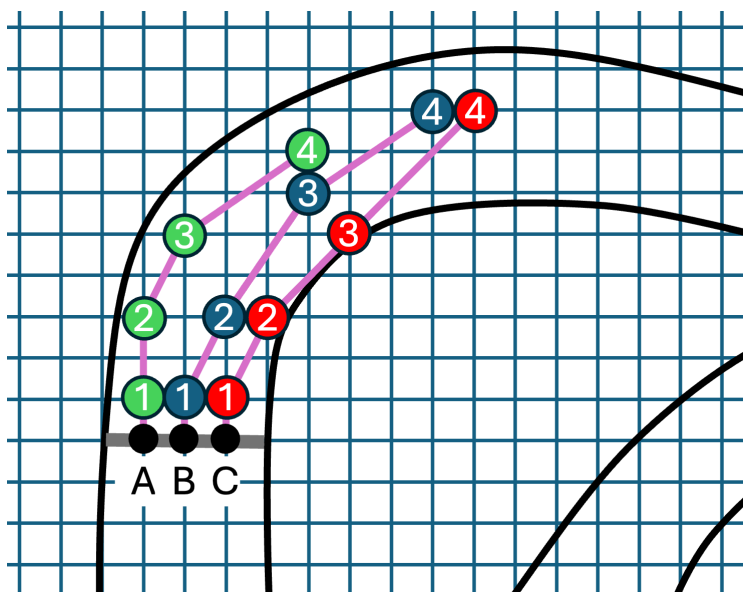
The possible actions are *accelerations* $a = (a_x, a_y)$, where both a_x and a_y can independently be $-1, 0$, or $+1$. The acceleration action updates the velocity ($v'_x = v_x + a_x$, $v'_y = v_y + a_y$) and then updates the location based on the new velocity ($x' = x + v'_x$, $y' = y + v'_y$).

The rewards are as follows:

- $+100$ for crossing the finish line in the correct direction.
- -100 for going off the track or crossing the finish line in the wrong direction. Both cases also lead to terminal states in which all actions have zero reward.
- -1 otherwise.

The discount factor is $\gamma = 0.5$ and the learning rate is $\alpha = 0.1$ unless otherwise specified.

The image below shows part of a racetrack and the first four timesteps of three different trajectories A, B, and C. Each starts on the finish line with velocity $(0,0)$ at time zero (the three black dots). The numbered circles show the positions at times 1, 2, 3, 4 for each trajectory.



Q6.1 (1 point) To make sure you have understood the rules, mark the trajectory that *violates* the rules, if any:

☒ A

☐ C

☐ B

☐ None of the above

Solution: Trajectory A has velocity $\begin{pmatrix} 1 \\ 2 \end{pmatrix}$ after timestep 2 and then $\begin{pmatrix} 3 \\ 2 \end{pmatrix}$, which is impossible.

(Question 6 continued...)

Q6.2 (1 point) Which trajectory, if any, is guaranteed to go off the track?

- ☐ A ☒ C
☐ B ☐ None of the above

Solution: The next velocity for C must be at least $\begin{pmatrix} 2 \\ 2 \end{pmatrix}$, which takes it off the track.

Q6.3 (2 points) Consider the “Stanford policy” that stops in the middle of the track and then circumnavigates a unit square forever. What is the utility (sum of discounted rewards) of this policy?

- ☐ A -1.0 ☒ C -2.0 ☐ E $-\infty$
☐ B -1.5 ☐ D -10.0 ☐ F None of the above

Solution: The total discounted return of an infinite sequence of -1 rewards is evaluated using an infinite geometric series:

$$U = \sum_{t=0}^{\infty} \gamma^t r = \frac{r}{1-\gamma} = \frac{-1}{1-0.5} = -2$$

Q6.4 (3 points) Suppose the track fits inside an $N \times N$ square. Which of the following is the tightest upper bound on the size of the state space? [Hint: consider the maximum achievable x and y velocities.]

- ☐ A $O(N^2)$ ☒ C $O(N^3)$ ☐ E None of the above
☐ B $O(N^{2.5})$ ☐ D $O(N^4)$

Solution: With maximum acceleration in the x or y direction, the distance travelled in T steps ($1 + 2 + 3 + 4 + \dots + T$) is $O(T^2)$, but cannot exceed N , so the maximum reachable velocity is $T = O(\sqrt{N})$; hence the state space is bounded by $O(N \cdot N \cdot \sqrt{N} \cdot \sqrt{N}) = O(N^3)$.

(Question 6 continued...)

Q6.5 (3 points) Oski decides to solve the racetrack problem using Approximate Q-Learning with a linear value function $Q(s, a) = w_1 f_1(s, a) + w_2 f_2(s, a)$, where $w_1 = 5$ and $w_2 = -30$, and

- $f_1(s, a)$ = anticipated magnitude of speed after taking the action.
- $f_2(s, a) = 1$ if the proposed action goes outside the track and 0 otherwise.

Suppose that in the present state s , a proposed action a has $f_1(s, a) = 4$ and $f_2(s, a) = 1$.

The agent takes action a , receives a reward $r = -100$, and transitions to a terminal state s_0 where all actions have zero reward.

With a learning rate $\alpha = 0.1$ and discount factor $\gamma = 0.5$, find the newly updated weight w_2 after the agent does a in s . (Hint: you may need to calculate the temporal difference error.)

☐ A -21

☐ C -40

☐ E -120

☒ B -39

☐ D -41

☐ F None of the above

Solution: First, calculate the current Q-value estimate:

$$Q(s, a) = w_1 f_1(s, a) + w_2 f_2(s, a) = (5)(4) + (-30)(1) = 20 - 30 = -10.$$

Next, calculate the temporal difference (target - prediction); note that the MDP is deterministic so there are no summations or probabilities:

$$\text{difference} = [r + \gamma \max_{a'} Q(s', a')] - Q(s, a) = [-100 + (0.5)(0)] - (-10) = -90.$$

Finally, update the active weight w_2 :

$$w_2 \leftarrow w_2 + \alpha [\text{difference}] f_2(s, a) = -30 + (0.1)(-90)(1) = -30 - 9 = -39.$$

(Question 6 continued...)

Q6.6 (2 points) Frustrated that the agent is driving too cautiously, Oski alters the reward function **for this subpart only**. He replaces the $r = -1$ step penalty with a positive reward equal to the updated speed: $R(s, a, s') = |v'_x| + |v'_y|$. Other rewards are unchanged.

Also, hoping to make winning more important, he sets γ to 0.99 **for this subpart only**.

Which of the following pathological behaviors is the agent most likely to learn?

- ☐ A The agent will safely drive to the finish line at the absolute maximum speed possible.
- ☒ B The agent will avoid the finish line and instead drive in a safe, high-speed loop forever.
- ☐ C The agent will immediately intentionally crash into a wall to terminate the episode and maximize expected utility.
- ☐ D The agent will learn to remain perfectly still ($v_x = 0, v_y = 0$) to accumulate infinite zero-penalties.
- ☐ E None of the above

Solution: This is a classic example of so-called **reward hacking** (probably better known as **reward mis-specification**, as the agent is doing just what we told it). It avoids crossing the finish line because that cuts off its supply of positive rewards and is worth only 100, whereas the loop policy has a value of roughly $\frac{v}{1-\gamma} = 100v$ where v is the average velocity on the loop.

Q6.7 (2 points) Reverting to the standard reward setup and discount factor, Oski notices that his Q-learning agent has found a safe but very slow policy to get to the finish line. It sticks with the policy and never tries anything else. What might work to fix this?

- ☐ A Decreasing the learning rate α over time
- ☒ B Using an ε -greedy action selection strategy
- ☐ C Increasing the discount factor γ closer to 1
- ☐ D Giving a bonus reward proportional to $N(s, a)$, the number of times a has been tried in s
- ☐ E None of the above

Solution:

- Decreasing α merely stops learning/updates; it does not force action exploration.
- ε -greedy forces the agent to take random actions with probability ε , ensuring the agent occasionally breaks out of its safe routine to explore higher velocities.
- Increasing γ changes the time horizon of rewards, but does not explicitly force the agent to try unvisited state-action pairs.
- This bonus reward has the opposite effect!

(Question 6 continued...)

Q6.8 (4 points) Unhappy with the performance of his simple feature-based Q-learning agent, Oski trains a very large and deep neural network to represent the Q-function, with the inputs being the state (x, y, v_x, v_y) and the action (a_x, a_y) . After a few million trials, the Q-function is driving really well!

Then he tries out the trained Q-function on a new track; much to his disappointment, it's a total failure. It soon veers across the track and crashes. Which of the following hypotheses and potential fixes are reasonable if Oski wants an agent that can drive well enough on any new track that comes along?

☐ A The network is too big and is memorizing rather than generalizing; a smaller network will work better.

☐ B Oski should train the Q-function on many different tracks and not just one; that will force it to generalize properly and work well for a new track.

☒ C The Q-function doesn't represent "how to drive well"; instead, it just approximates the Q-values for this particular track, which tell it what to do in each state. On a different track, it can be in the same state but a completely different action is needed.

☐ D The problem is with the input representation. The network needs an additional input indicating the identity number of the track; then multi-track training will work and the Q-function will generalize to new tracks.

☒ E The problem is with the input representation. The network will work better with an image of the track as an additional input; then multi-track training will work and the Q-function will generalize to new tracks.

☐ F None of the above

Solution: A sounds sensible but fails: the optimal action simply isn't a function of the given state if the track can change.

B fails for the same reason. The input doesn't tell the agent which track it is on and hence it cannot act accordingly.

C is a correct diagnosis.

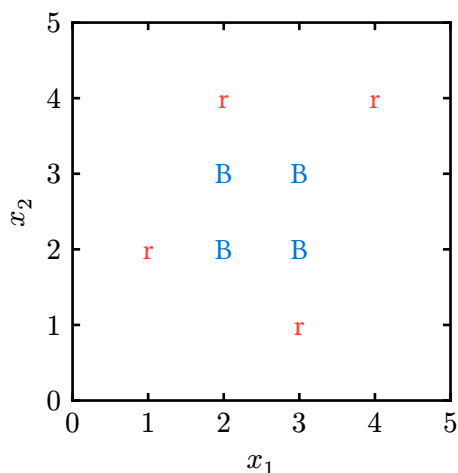
D seems to fix the problem with B, but it still fails. With each new track it has to start from scratch because the identity input doesn't "describe" the track, it just indexes it.

E is a plausible solution: the neural network should learn to extract driving relevant features from the current state and the track image, such as "distance to next bend" and so on.

Q7 Five Nights with Machine Learning**(10 points)**

You are a security guard working the night shift at a local pizzeria. Each location (x_1, x_2) at the pizzeria has one animatronic (robot animal) present. Each animatronic is either a big bear (B) or a little rabbit (r).

For Q7.1 to Q7.3, you visit some locations and record the type of animatronic at each location. You graph the data in the following plot.



Q7.1 (1 point) It may be possible to use machine learning to predict what animal appears in each unexamined location. We can try training a single-layer perceptron, with input x_1, x_2 and output labels $+1$ for a bear and -1 for a rabbit. What happens on this data set, assuming a constant learning rate?

- ☐ A The weights will always converge to a perfect classifier
- ☐ B The weights will converge but with some misclassifications
- ☐ C The weights will only converge if the learning rate is 0.5
- ☒ D The weights are not guaranteed to converge
- ☐ E None of the above

Solution: Perceptrons converge on linearly separable data, or they require an adaptive learning rate. There may be an initialization edge case that could lead to convergence.

Q7.2 (1 point) While training, your perceptron incorrectly classifies location $(3, 2)$ as a rabbit. What happens to w_1 , the weight corresponding to feature 1? Assume that your learning rate is positive.

- ☒ A Increases
- ☐ B Decreases
- ☐ C Stays the same

Solution: We update the weight by the product of learning rate α (positive), the error $(+1 - (-1) = 2)$, and the feature value (3). This is positive.

(Question 7 continued...)

Q7.3 (2 points) You want to add a third feature, x_3 to help us better classify the data using a perceptron. Which of the following definitions for x_3 would allow a perceptron to classify the data perfectly?

☐ A $x_3 = x_1 + x_2 + 2.5$

☐ D $x_3 = (x_1 + x_2)^2 - 2.5$

☐ B $x_3 = x_1 - x_2$

☐ E $x_3 = 2.5$

☒ C $x_3 = (x_1 - 2.5)^2 + (x_2 - 2.5)^2$

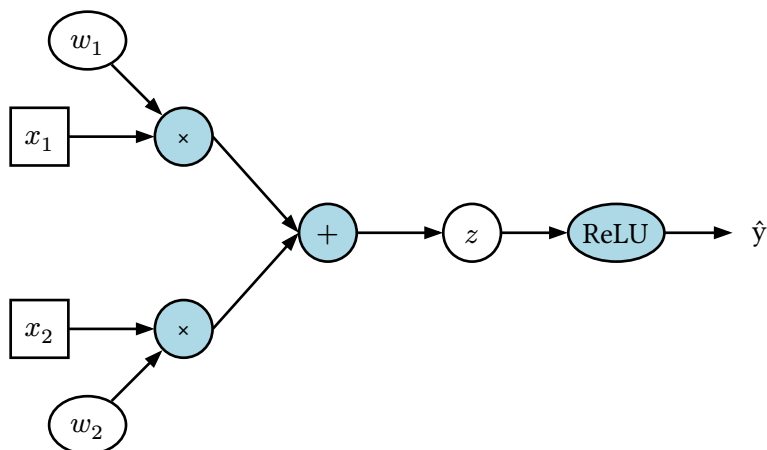
☐ F None of the above

Solution: The feature in C is the squared distance from (2.5, 2.5). A threshold of 1 on this feature value separates bears and rabbits with a circular boundary centered at (2.5,2.5). D only allows circles centered on (0,0), which doesn't help.

(Question 7 continued...)

For Q7.4 to Q7.6, you collect the heights of the animatronics at each location. You decide to build a neural network to predict the height of the animatronic at a location.

The neural network takes in two inputs, x_1 and x_2 , representing the location. The network outputs $\hat{y}$, which is the predicted height. The computation graph is shown below.



Note that $\hat{y} = \text{ReLU}(z)$. We use the loss function $L(y, \hat{y}) = \frac{1}{2}(y - \hat{y})^2$

Q7.4 (2 points) Let the weights be $w_1 = 1$ and $w_2 = -2$. Calculate $\hat{y}$ for the input $(x_1, x_2) = (1, 1)$.

0

Solution: $\text{ReLU}(1 \cdot 1 + -2 \cdot 1) = 0$

Q7.5 (3 points) Now, we want to calculate $\frac{\partial L}{\partial w_1}$ during the backward pass. What partial derivatives are needed?

☐ A $\frac{\partial L}{\partial x_1}$

☐ F $\frac{\partial \hat{y}}{\partial L}$

☐ K $\frac{\partial x_1}{\partial z}$

☐ B $\frac{\partial L}{\partial x_2}$

☒ G $\frac{\partial \hat{y}}{\partial z}$

☐ L $\frac{\partial x_2}{\partial z}$

☒ C $\frac{\partial L}{\partial \hat{y}}$

☐ H $\frac{\partial z}{\partial \hat{y}}$

☒ M $\frac{\partial z}{\partial w_1}$

☐ D $\frac{\partial x_1}{\partial L}$

☐ I $\frac{\partial z}{\partial x_1}$

☐ N $\frac{\partial w_1}{\partial z}$

☐ E $\frac{\partial x_2}{\partial L}$

☐ J $\frac{\partial z}{\partial x_2}$

☐ O None of the above

Solution: Chain rule: $\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_1}$

(Question 7 continued...)

Q7.6 (1 point) Now, we want to use the partial derivative $\frac{\partial L}{\partial w_1}$ to update the value of w_1 to help our neural network predict better. Fill in the operator that best completes the update rule. α is the learning rate.

$$w_1 \leftarrow w_1 \text{ ______ } \alpha \frac{\partial L}{\partial w_1}$$

Ⓐ +

Ⓑ −

Ⓒ ×

Ⓓ ÷

Solution: We want to minimize loss, so we do gradient **descent**, and we subtract.